

Introduction to Red Black Trees

- ◆ Trees offer many many advantages – especially for quick inserting and reasonably quick deleting.
- ◆ Binary tree searches provide outstanding performance in almost all instances
- ◆ ☐ **Big Problem: unbalanced trees** ☐
- ◆ This will significantly reduce search times and more...
 - Unbalanced trees may easily arise if incoming data is sorted or nearly sorted (up or down!)
 - We need balanced or nearly-balanced trees.

Red Black Trees - RBTrees

- ◆ We will not cover this entire chapter, but we will cover the most salient parts
- ◆ Red Black Trees is an approach to
 - restructure unbalanced trees and moreso, to
 - maintain balanced trees when we start to
 - insert or
 - delete nodes.

Red Black Trees - RBTrees - more

- ◆ Not trivial to understand ...
 - All nodes will either be 'red' or 'black' with constraints!!
- ◆ No code is found in this chapter.
- ◆ Used in 2-3-4 and 2-3 trees next chapter to assist in balancing multi-way trees – especially for external storage.
- ◆ **Strongly encourage exercising the applet, although book provides close coverage.**

Insertions into Red Black Tree

◆ Top Down Insertions:

- Some **structural changes** (such as rotations, flips, color changes, etc.) may be made **as part of the search routine.**

◆ Bottom up Insertions:

- Look for the proper place to add the node
- **Then, we proceed up the tree making structural changes.**
- Less efficient than top-down insertions.

Balanced and Unbalanced Trees

- ◆ Easy to see how a tree becomes unbalanced.
 - Think all nodes ascending (or descending) and resulting structure.
- ◆ An unbalanced tree results in the same structure as a linked list.
 - Still would be going from node to node to node...
 - Would provide $O(n)$ performance for searching, insertions, etc.
 - Consider: sequential search (array or linked list) for 1000 elements □ average search time of 5000; **But**: binary search tree: **max** is 10 **if tree is balanced** (the only way to go...)

Balanced and Unbalanced Trees

- ◆ Very important to have tree balanced
- ◆ As it turns out, when input is random, the binary search tree development will often be reasonably 'okay.'
- ◆ Must **watch** out for a **key values** that represent extremes.
- ◆ We need trees that have approximately the same number of descendents on both sides of all nodes.
- ◆ In RBTrees, **we constantly balance during** the insertion and deletion process.
 - That is, as we insert and delete, we will **balance as part of the procedure.**
 - During these processes, certain RB characteristics **must be maintained.** If not, structural changes take place as part of insertion and deletion.

Red Black Tree Characteristics

- ◆ Lots of rules and characteristics....
- ◆ **All** nodes are colored (red or black)
 - (Nice for University of Georgia fans!)
- ◆ **During insertions and deletions** all the rules (coming) are **checked** to preserve the balance of the tree.
 - **This means that all the Red Black rules must be verified along the way.**

Red Black Rules – The Bigees...

- ◆ These are **essential** in understanding the RB Tree concepts and how the algorithms work (and hence how the applet works).
 1. Every node is either red or black
 2. The root is always **black**
 3. If a node is **red**, its children must be **black** (converse not true).
 4. **Every path** from root to a leaf (or to a null child – important later) must contain the **same number of black nodes**.
- ◆ (number of black nodes on a path from root to node is called the black height.)

Violations as We Process a Tree

- ◆ It is permissible (actually **REQUIRED**) to fix the violations as we insert (and delete) – if we want to keep the tree balanced!
- ◆ We can flip (change colors... more coming)
- ◆ We can rotate left and rotate right (RoL, RoR)
- ◆ Running the Applet
Book as description of the buttons (several)
 - Start, Insert, Delete, Flip, RoL, RoR, R/B.
 - There are also text messages telling the applet user when rules have been violated.

Actions

◆ The Flip button:

- When arrow is placed on a parent and Flip is depressed, color of root is flipped; color of children is flipped (all children will be same).
 - If node is black and we flip to red, then its children must be black. (rule 3)
- Changes colors of **at most** three nodes at once

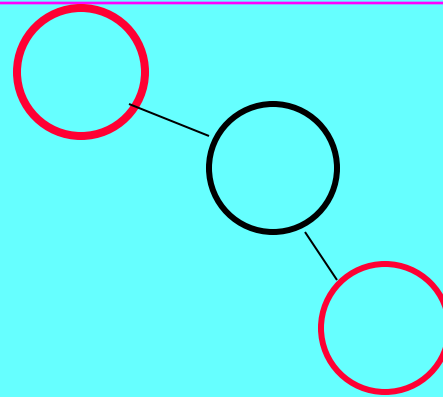
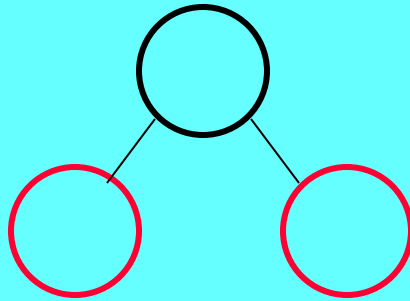
◆ Rotate Left:

- Position yourself on the topmost node of group to be rotated.
- RoL – topmost node **MUST** have a right child.
- RoR – topmost node **MUST** have a left child.
- R/B – Changes a node's color (single node).

Experiments

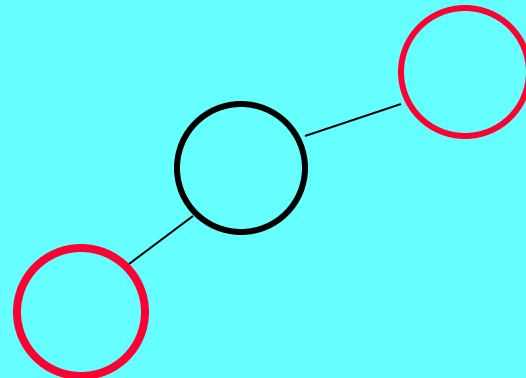
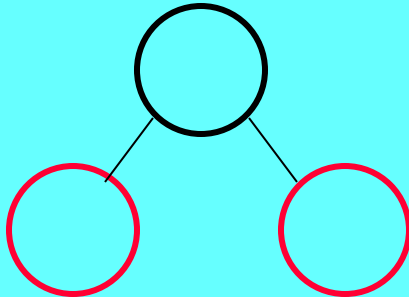
- ◆ **Part** of the process of adding a node is to verify that the RB rules are accommodated.
- ◆ **When we insert a node**, we will start with a red node because this will cause fewer problems later.
 - E.g. doesn't change black height...and more.

Ro
R



Remember: RoR requires the node to have a non-null left subtree.

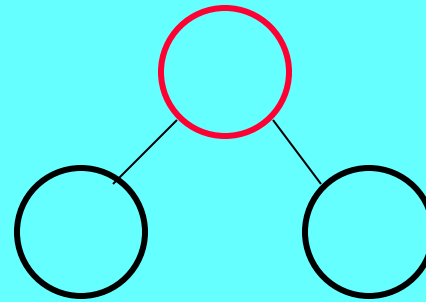
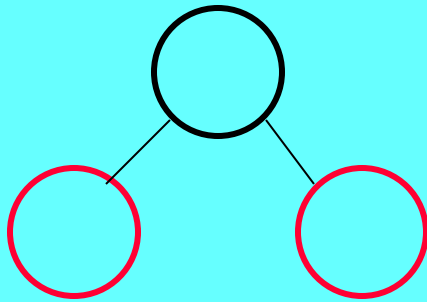
Ro
L



Remember: RoL requires the node to have a non-null right subtree.

Remember: ¹² the root node must be Black!!

Color Flips



Cannot do here because root cannot be red.
Assumption above: we are looking at the root.

Otherwise, it'd be okay...

Message here: Can't Insert.

Again, normally okay, unless node is the tree root.

Unbalanced Tree

- ◆ Definition of Unbalanced Tree: when one path (from root to leaf nodes) differs from another by **more than one level**. (two or more levels).
- ◆ **If we insert** a node and the tree becomes unbalanced, we need to **balance** the tree.
 - □ Since we traditionally **add Red nodes**, the typical message will be parent and child are both red.
 - (Rule 3 violated - If node is red, children must be black.
- ◆ But sometimes just flipping might result in black heights **differing**!
 - Can become complicated as we often may need to **both rotate and flip...**

Nice to Knows...

- ◆ □ One **canNOT** create an **unbalanced tree** if all of the red-black rules are accommodated.
- ◆ □ Equivalently, following the four Red Black rules **ensures** that the tree will be balanced.
- ◆ □ “If one path is **more than** one node longer than another, it **must either** have more black nodes, **violating Rule 4**, or it must have two adjacent red nodes, **violating Rule 3**.”
 - **Rule 4:** Every path from root to a leaf (or to a null child – important later) must contain the **same number** of **black** nodes.
 - **Rule 3:** If a node is red, its children must be black (converse not true).

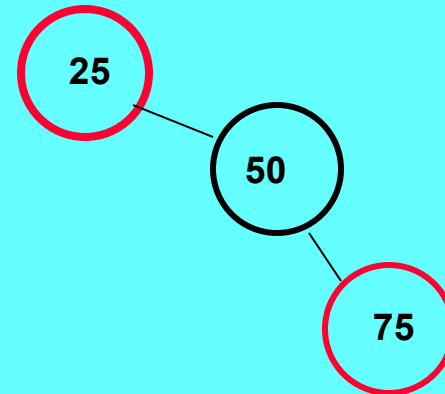
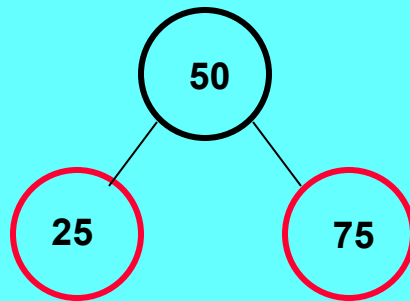
Rotations

- ◆ These can become complicated.
- ◆ Rotations:
 - 1. must raise some nodes (that is, raise to a higher level) and lower other nodes **to balance the tree**, and
 - 2. must ensure that the **characteristics** of a **binary search tree** are not violated.
- ◆ We will look at
 - 1. Simple Rotations
 - 2. Cross Over Nodes
 - 3. Moving subtrees

1. Simple Rotations

- ◆ It is actually the relationship between nodes that changes: a parent becomes a child...
- ◆ Rotations don't necessarily involve the root.
- ◆ Any node can be the center of rotation; that is, any node of any sub-tree can be the center of rotation.
- ◆ Just remember: a RoR requires the top node to have a left child and conversely.
- ◆ Simple Rotation: a repeat slide – but for completeness:

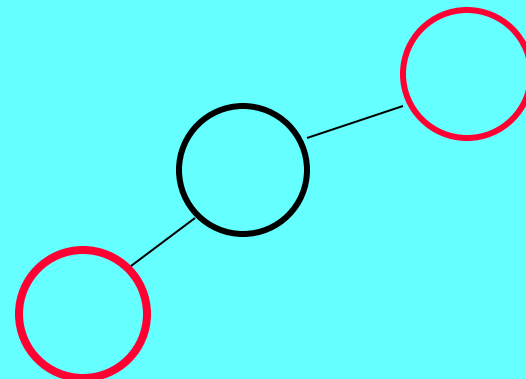
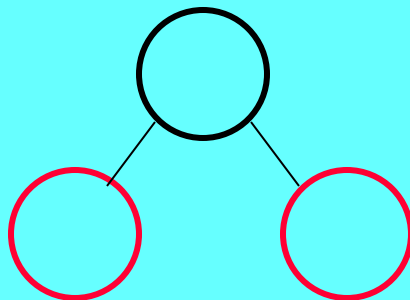
Ro
R



Remember: RoR requires the node to have a non-null left subtree.

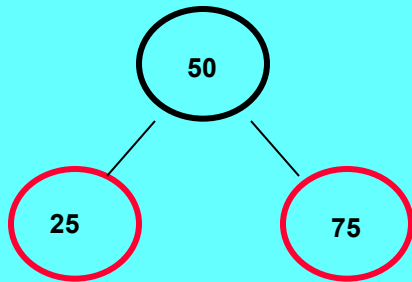
~~Of course, this cannot remain this way! Why not??~~

Ro
L

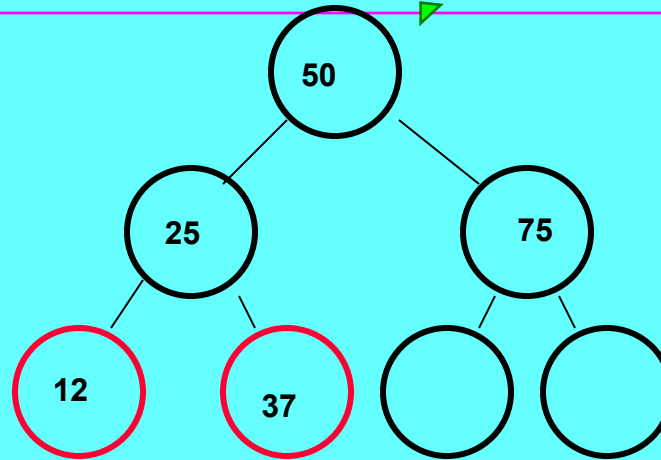


Remember: RoL requires the node to have a non-null right subtree.

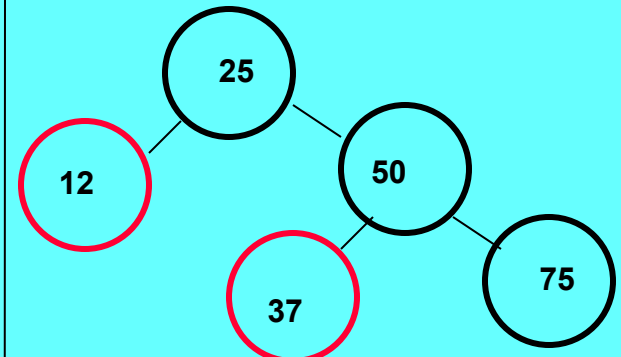
2. Cross-Over Rotations – a bit more complicated:



**RB Tree:
from last
slide**



**Null
nodes**



Two actions here: 1. Insert 12; 12 (new) is red. Get message: Needs color flip (25 originally is red; Click Flip button; Parent and child swap colors. OK

2. Insert 37. Successful. We end up with the arrangement in the center.

1. Every node is either red or black
2. The root is always **black**;
3. If a node is **red**, its children must be **black** (converse not true).
4. **Every path** from root to leaf (or to a null child – important later) must contain the **same number of black nodes**.

Now try a rotation right (placing arrow in applet on root).

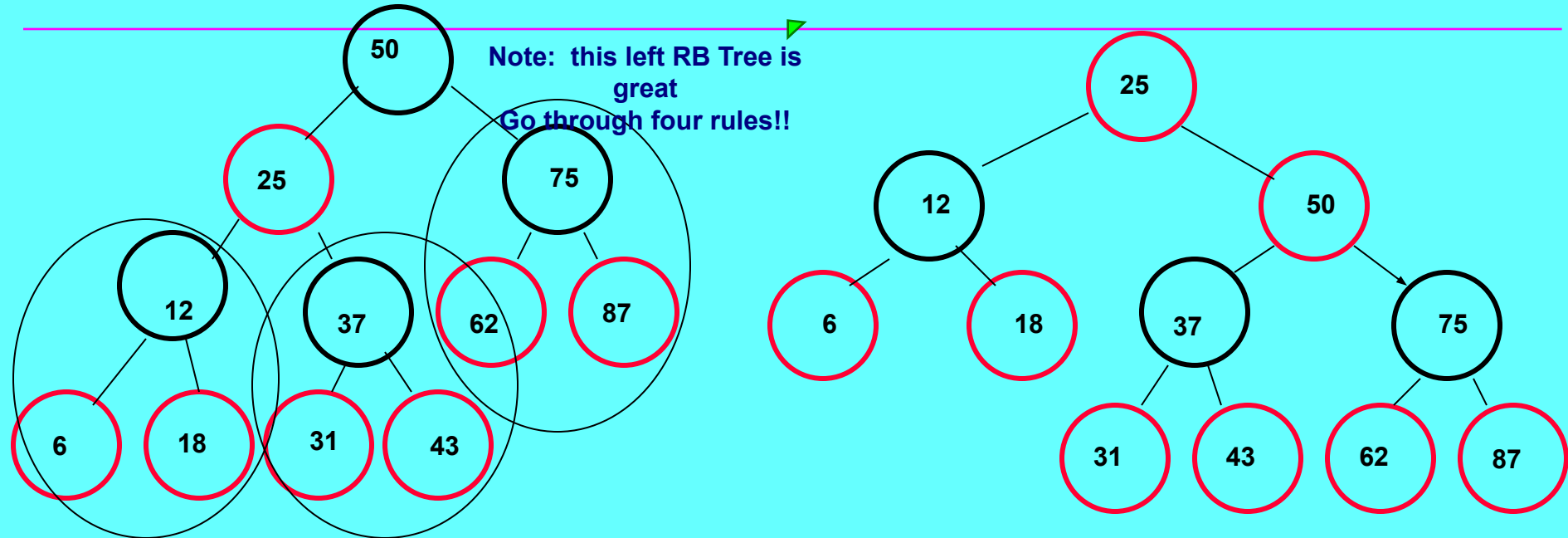
Note: 37 moved across from right child of 25 to left child of 50!!

Had to be done in keeping this a binary search tree.!

But Rule 4 (Every path from root to leaf (or to a null child) must contain = number black nodes.)

For now, in middle figure, 37 is called an **inside grandchild** (and 12 is called an *outside grandchild*)!! If an inside grandchild is a child of a node that is 'going up,' it is always disconnected from parent and reconnected to its former grandparent! How 'bout that!!

Subtrees Moved – Entire Subtrees can be moved!



With 50 as the root, we will insert (in order) 25, 75, 12, 37, 62, 87, 6, 18, 31, and 43.

We must click Flip whenever we get a Can't Insert Needs Color Flip.

We arrive at the figure on the left. (All rules okay?) **Verify!!!!**

Rotate Right, and we get the second figure.

Note: 50 went to the right; root's left child is now root. Entire subtree with 12 moved up.

□ Entire subtree of 37 crossed over! (inside grandchild!!)

But there are problems (look at root...) (root must be black (Rule 1) and it is not, and for a red node (like 50), children must be black (Rule 2))

Detailed fix is found ahead in chapter...

□ But note the **relation** between the nodes in the sub-trees is unchanged by the rotation.

Entire sub-tree moves in a rotation²⁰ no matter the size of the sub-tree.

Insertions

- ◆ I will not cover the section on Inserting a New Node.
- ◆ □ We are not using RBTrees in our projects, but I DO want you to know the rules and a number of their characteristics!
 - But flips and rotations are encountered algorithmically once a place for the node is determined (via standard binary search tree insert procedures).
 - There is more information on the inside and outside grandchildren.
 - Sometimes they rotate left; sometimes right; sometimes they change color.
 - These structural changes are implemented all the way down to the new node you have inserted to **ensure the RB rules are accommodated and hence the tree is balanced!**

Deletions

- ◆ Deletions in binary trees are quite involved.
- ◆ Normally, in practice, designers and programmers merely 'mark' the node as logically deleted (but physically it is still present).
 - OK for small number of deletions
 - Can then later delete, if needed, 'off line.'

Efficiency of Red Black Trees

- ◆ Search is very fast, of course, because the resulting tree is balanced.
- ◆ All $O(\log_2 n)$ time! Super.
 - Small storage issue per node to include an `isBlack()` or `isRed()` boolean method. No big deal.
- ◆ Insertions and Deletions require additional time due to requirements to do color flips and rotations all the way down to the insertion point.
- ◆ Because most insertions require an average of a single rotation, insertion still takes about $O(\log_2 n)$ time but a bit slower than insertion in the ordinary binary tree.

Implementation of Red Black Trees

- ◆ Algorithm is complex.
- ◆ Steps are outlined for the 'interested' implementer.
- ◆ The really nice thing about Red Black Trees is that we can be assured they will be balanced!
- ◆ This is essential for good performance!!

AVL Trees

- ◆ Another kind of balanced tree.
- ◆ AVL trees also store the difference between the heights of a node's left and right subtree.
 - (cannot be > 1 , of course. Won't be balanced.)
 - AVL trees do require some of the same operations (rotations and moves...)
 - But: **AVL trees are guaranteed to be balanced!**
- ◆ Now, on to Multi-Way trees!!

Study:

- ◆ Chapter Summary
- ◆ Questions (answers are in Appendix C)
- ◆ Know the four RB Rules. If you go over the materials a few times, they become automatic.